Improving CLIP Training

Fall 2026: Machine Learning Course Project Description

1 Introduction

In this project, we will have a competition to solve a challenging problem, namely optimizing global contrastive loss for bimodal contrastive self-supervised learning.

Self-supervised learning (SSL) for pre-training deep neural networks has emerged to be a popular paradigm to learn data representations that generalize across downstream tasks. It has demonstrated effectiveness in multiple areas, including Natural Language Processing [10, 2, 8] and Computer Vision [3, 18, 9]. A simple yet effective framework of SSL is Contrastive Learning (CL), which has laid the foundation for state-of-the-art SSL models due to its effectiveness [1, 5, 13, 7]. CL aims to push the similarity scores between "positive" pairs (e.g., augmented views of the same image) to be higher than that between "negative" pairs (e.g., augmented views from different images), which has great promises in leveraging large amount of unlabelled data [4, 11]. Moreover, CL has been extended to a broader scope, e.g., bimodal image-text SSL [17, 11], where images and language text descriptions can be regarded as multi-modal views of the same underlying concept. The well-known CLIP [11] method shows that models learned from millions of image-text pairs can attain impressive recognition performance for a wide range of visual understanding tasks.

While the great performance of CLIP and its alternatives have been demonstrated on popular benchmarks, contrastive self-supervised learning for large-scale bimodal image-text SSL remain a challenging problem due to slow convergence and other issues. In this project, you are asked to propose your solutions to accelerate the optimization of global contrastive loss, and to make the learned model have better performance.

2 Global Contrastive Loss and SogCLR

In [16], we have proposed a global contrastive loss (GCL). Below, we first introduce GCL and then talk about SogCLR for optimizing it.

Notations: Let $\mathcal{D} = \{(x_1, z_1), \ldots, (x_n, z_n)\}$ be a dataset of n image-text pairs, where x_i denotes an image and z_i denotes it corresponding text. Let w denote the parameters of an image encoder and a text encoder jointly, which are the parameters that need to be optimized. Let $h_i(w)$ denote the (normalized) output image feature vector by feeding image x_i to the image encoder, and $e_i(w)$ denote the (normalized) output text feature vector by feeding text z_i to the text encoder.

Task: Train an image and text encoder w using the training dataset $\mathcal{D}$.

2.1 CLIP

The original CLIP model was trianed by minimizing mini-batch based contrastive loss. Denote by $\ell_1(x_i, z_j; \tau) := \exp(\frac{h_i(w)^T e_j(w) - h_i(w)^T e_i(w)}{\tau})$, and $\ell_2(z_i, x_j; \tau) := \exp(\frac{e_i(w)^T h_j(w) - e_i(w)^T h_i(w)}{\tau})$. where $\tau > 0$ is the temperature parameter. The original method of CLIP is to update the model according to the gradient of a mini-batch contrastive loss that is defined over sampled mini-batch data. To this end, a random mini-batch of m image-text pairs $\mathcal{B} = \{(x_1, z_1), \ldots, (x_B, z_B)\}$ are first

sampled. For each image-text pair (x_i, z_i), the mini-batch contrastive loss is defined below:

$$L(w, \tau, x_i, z_i, \mathcal{B}) = \log\left(\frac{1}{|\mathcal{B}_i|}\sum_{z_j \in \mathcal{B}_i} \ell_1(x_i, z_j; \tau)\right) + \log\left(\frac{1}{|\mathcal{B}_i|}\sum_{x_j \in \mathcal{B}_i} \ell_2(z_i, x_j; \tau)\right).$$

This loss contrasts the similarity score between corresponding image-text pairs (x_i, z_i) and non-corresponding image-text pairs (x_i, z_j) and (z_i, x_j) in the same batch. Then the gradient w.r.t. w is computed based on the following local contrastive loss for each image-text pairs.

$$L(w, \tau, \mathcal{B}) := \frac{1}{m}\sum_{x_i \in \mathcal{B}} L(w, \tau, x_i, \mathcal{B}).$$

In the original method of CLIP training, the temperature τ is also treated as a parameter and optimized in the same way as w.

2.2 A Global Contrastive Objective

The local contrastive loss defined over the mini-batch samples hides the complexity for contrastive learning, which renders CLIP sensitive to the mini-batch size. To address this issue, we have proposed a global contrastive objective in [16].

$$L_1(w, \tau, x_i, \mathcal{D}) = \tau \log\Bigg(\underbrace{\varepsilon + \frac{1}{|\mathcal{T}_i^-|}\sum_{z_j \in \mathcal{T}_i^-} \ell_1(x_i, z_j; \tau)}_{g_1(w, x_i, \mathcal{D})}\Bigg)$$

$$L_2(w, \tau, z_i, \mathcal{D}) = \tau \log\Bigg(\underbrace{\varepsilon + \frac{1}{|\mathcal{I}_i^-|}\sum_{x_j \in \mathcal{I}_i^-} \ell_2(z_i, x_j; \tau)}_{g_2(w, z_i, \mathcal{D})}\Bigg),$$

where ε is a hyperparameter introduced to increase the generalization, $\mathcal{T}_i^- := \{(z_j) : j \neq i\}$ denotes negative texts of image x_i and $\mathcal{I}_i^- := \{(x_j) : j \neq i\}$ denotes negative images of text z_i . Different from CLIP which contrasts image-text pairs within the same **batch**, the above loss contrasts image-text pairs within the **whole dataset**. Based on this loss, the following **global contrastive objective (GCO)** is proposed

$$L(w, \tau, \mathcal{D}) := \frac{1}{n}\sum_{x_i \in \mathcal{D}} L_1(w, x_i, \mathcal{D}) + \frac{1}{n}\sum_{z_i \in \mathcal{D}} L_2(w, z_i, \mathcal{D}). \tag{1}$$

2.3 SogCLR, iSogCLR

Below, we introduce two methods that we developed for improving the training of CLIP models with a small mini-batch size. To address the large batch-size issue of CLIP, we have proposed a memory-efficient stochastic algorithm for (1) without suffering from a large optimization error depending on the batch size. The key idea of SogCLR is the introduction of the following two moving average sequences

$$\begin{aligned} u_{1,i,t} &= (1-\gamma)u_{1,i,t-1} + \gamma g_1(w_t, x_i, \mathcal{B}_{1t}^-), \\ u_{2,i,t} &= (1-\gamma)u_{2,i,t-1} + \gamma g_2(w_t, z_i, \mathcal{B}_{2t}^-). \end{aligned} \tag{2}$$

where $\mathcal{B}_t$ is the mini-batch, $\mathcal{B}_{1t}^-$ and $\mathcal{B}_{2t}^-$ denote the negative texts and images for x_i and z_i in the mini-batch, respectively. Then, we update the parameters with the following gradient estimator

$$G_t = \frac{\tau}{m}\sum_{x_i \in \mathcal{B}_t} \frac{1}{\varepsilon + u_{1,i,t}} \cdot \nabla g_1(w_t, x_i, \mathcal{B}_t) + \frac{\tau}{m}\sum_{z_i \in \mathcal{B}} \frac{1}{\varepsilon + u_{2,i,t}} \cdot \nabla g_2(w_t, z_i, \mathcal{B}_t). \tag{3}$$

Finally, we can update the model parameters w_{t+1} by using any optimizer (e.g., Adam-W). The detailed steps are summarized in Algorithm 1, which is referred as SogCLR to emphasize that we aim to optimize the global contrastive objective. The pseudocode is provided in Algorithm 2 For motivation and analysis of the algorithm, please refer to [16].

Algorithm 2: PyTorch-style Pseudocode for SogCLR

```
# model1 and model2: image encoder and text encoder respectively
# tau: temperature
# n: data size
# idx: indices for image-text pairs in mini-batch
# u1, u2: 1d tensors with shape (n,1) by zero initialization
# gamma: parameter for maintaining moving averages of u1 and u2

# dynamic contrastive loss (mini-batch)
def dcl(feat1, feat2):
    m = feat1.shape[0]
    sim_neg = matmul(feat1, feat2.T)
    sim_pos = sum(mul(feat1, feat2))
    sim_diff = sim_neg - sim_pos
    mask = 1 - eye(m)
    g = sum(exp(sim_diff / tau), dim=-1) / (m - 1)
    return g

for img, txt, idx in dataloader:
    h, e = model1(img), model2(txt)
    loss1 = dcl(h, e)
    loss2 = dcl(e, h)
    u1[idx] = (1 - gamma) * u1[idx] + gamma * loss1.detach()
    u2[idx] = (1 - gamma) * u2[idx] + gamma * loss2.detach()
    loss = mean(loss1 + loss2)
    loss.backward()
    optimizer.step()

```

Algorithm 1: SogCLR

Input: $w_0 \in \mathbb{R}^d, u_0 \in \mathbb{R}^n$;
for $t = 1, \ldots, T$ **do**
  Draw a batch of m image-text pairs denoted by $\mathcal{B}_t = \{(x_i, z_i)\}_{i=1}^m$;
  for $(x_i, z_i) \in \mathcal{B}_t$ **do**
    Compute $g_1(w, x_i, \mathcal{B}_{1t}^-)$ and $g_2(w, z_i, \mathcal{B}_{2t}^-)$;
    Update $u_{1,i,t}$ and $u_{2,i,t}$ according to (2);
  Compute the gradient estimator G_t according to (3);
  Update $v_t = (1-\beta)v_{t-1} + \beta G_t$;
  Update $w_{t+1} = w_t - \eta v_t$ (or use Adam-style update);

The iSogCLR algorithm is an improvement of SogCLR by optimizing the temperature parameters. Inspired by distributionally robust optimization, the problem is formulated as

$$\min_{w,\tau_1,\tau_2} \frac{1}{n} \sum_{x_i \in \mathcal{D}} L_1(w, \tau_{i1}, x_i, \mathcal{D}) + \frac{1}{n} \sum_{z_i \in \mathcal{D}} L_2(w, \tau_{i2}, z_i, \mathcal{D}) + \rho \sum_{i=1}^{n} (\tau_{i1} + \tau_{i2}) \tag{4}$$

where $\tau_1 = (\tau_{11}, \ldots, \tau_{n1})$ and $\tau_2 = (\tau_{12}, \ldots, \tau_{n2})$.

3 Requirements

The goal of this course project is to ask you to develop better algorithms to train CLIP models. We will fix some parameters in the project:

- Data: you will use a 100k subset of the Conceptual Captions 3M (CC3M) dataset for training. For validation, you will use the MSCOCO validation dataset and the ImageNet validation dataset, which will be used to evaluate the model's retrieval and zero-shot classification performance respectively. For testing, we will use two different subsets from MSCOCO and

ImageNet for retrieval and zero-shot classification. The training and validation datasets will be provided [1], while the testing datasets will not be released. Note that you are allowed to do training only on the training dataset, and the use of other external datasets in the training stage is not allowed.

- Metric: The evaluation metric is the average of Image-to-Text Recall at the position 1, Text-to-Image Recall at the position 1 on the retrieval dataset and the Top 1 Accuracy on the classification dataset. You are expected to select models on the validation datasets. Your submitted model will be evaluated on the testing datasets and ranked accordingly.
- Model: you will use a ResNet-50 [6] pretrained on ImageNet from timm library [14] as the image encoder, and a DistilBERT [12] pretrained on BookCorpus [19] and English Wikipedia from huggingface library [15] as the text encoder.
- Hyperparameters: training batch size is fixed to 128, and the number of training epochs is fixed to 30.

We have provided a codebase that implements the SogCLR and iSogCLR in Pytorch, which is available here `https://colab.research.google.com/drive/1FTF-cTcW11Gyrwu8uhTZOXgLsjp49Z9W?usp=sharing`. You are free to make modification to any components and also explore other codebase (e.g., FastCLIP, TempNet) as long as you use the same settings as specified above.

4 Requirement

- You need to compare at least 2 different optimizers.
- You need to compare at least 3 different loss functions.
- In the report, you need to report the performance of your trained models on the provided validation datasets of MS-COCO and ImageNet in terms of averaged recalls and zero-shot classification
- You are asked to submit your code for training and your trained model so that we can load your model for evaluation.

Grading We will evaluate the model by the zero-shot top-1 accuracy on the testing classification dataset and image-to-text retrieval and text-to-image retrieval performance by using recall@1 on the testing retrieval dataset. Finally, we will use the average performance to rank different groups. The top 6 groups will get a certificate (1st, 2nd, 3rd, 3 honoral mentions). We will use the following criteria for grading:

- Breadth of experimental comparison (30')
- Writing of report (45')
- Presentation (25')
- Novelty of your ideas (+10')

We will follow the timeline below:

- Project Presentation (upload a 10-minutes video).
- Final Project Report (5 + pages), including abtract, introduction, related work, proposed work, experiments with detailed results, and conclusion. You can use the provided latex template. If you use word, please use the font size of 11pt and 1 inch margin at all sides. In the final report, every team should include a description of team members' contribution. If any team member's contribution is not clearly described, he/she will get a zero score.

[1] Training and validation datasets: `https://drive.google.com/drive/folders/1Wh-X0YHUoJCRdIS2hVFtRC6xU5JToqvz?usp=sharing`. Usage is provided in the codebase.

References

[1] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, pages 1597–1607, 2020.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[4] Priya Goyal, Mathilde Caron, Benjamin Lefaudeux, Min Xu, Pengchao Wang, Vivek Pai, Mannat Singh, Vitaliy Liptchinsky, Ishan Misra, Armand Joulin, et al. Self-supervised pretraining of visual features in the wild. *arXiv preprint arXiv:2103.01988*, 2021.

[5] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the 33rd IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9729–9738, 2020.

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[7] Zhicheng Huang, Xiaojie Jin, Chengze Lu, Qibin Hou, Ming-Ming Cheng, Dongmei Fu, Xiaohui Shen, and Jiashi Feng. Contrastive masked autoencoders are stronger vision learners. *arXiv preprint arXiv:2207.13532*, 2022.

[8] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942*, 2019.

[9] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021.

[10] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[11] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763, 2021.

[12] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[13] Nenad Tomasev, Ioana Bica, Brian McWilliams, Lars Buesing, Razvan Pascanu, Charles Blundell, and Jovana Mitrovic. Pushing the limits of self-supervised resnets: Can we outperform supervised learning without labels on imagenet? *arXiv preprint arXiv:2201.05119*, 2022.

[14] Ross Wightman. Pytorch image models. `https://github.com/rwightman/pytorch-image-models`, 2019.

[15] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[16] Zhuoning Yuan, Yuexin Wu, Zi-Hao Qiu, Xianzhi Du, Lijun Zhang, Denny Zhou, and Tianbao Yang. Provable stochastic optimization for global contrastive learning: Small batch does not harm performance. In *International Conference on Machine Learning*, pages 25760–25782. PMLR, 2022.

[17] Yuhao Zhang, Hang Jiang, Yasuhide Miura, Christopher D Manning, and Curtis P Langlotz. Contrastive learning of medical visual representations from paired images and text. *arXiv preprint arXiv:2010.00747*, 2020.

[18] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable detr: Deformable transformers for end-to-end object detection. *arXiv preprint arXiv:2010.04159*, 2020.

[19] Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *The IEEE International Conference on Computer Vision (ICCV)*, December 2015.